Agent 开发工具的分层定位与企业选型调研

七个"都能做 Agent"的工具：LangChain/LangGraph、Pydantic-AI、OpenClaw、Nous Hermes-Agent、DeepSeek Harness (dsh)、Codex CLI、Pi

调研时间窗：**2026-11-03**（单一时点快照）｜方法：GitHub API 直连核实 + 官方文档精读 + 三代表工具源码级深读

目录

0	结论摘要（含七工具总览矩阵）
1	存在性核实
2	分层定位与关系图　2.1 三层定位表与归层依据　2.2 关系图与 MCP/ACP 方向明细　2.3 跨层直比为何误导与比较方法
3	架构与核心抽象（能力状态四态 · 七维度统一定义）　3.1 LangGraph　3.2 Codex CLI　3.3 OpenClaw　3.4 其余五工具简述
4	企业落地考量　4.1 安全　4.2 合规　4.3 成本　4.4 维护成熟度　4.5 国内可用性
5	场景选型建议（5.1 内网合规问答 · 5.2 高并发客服 · 5.3 研发助手 · 5.4 长期记忆助理）　5.5 题目 2 选型论证
6	附录　6.1 调研方法　6.2 "查无实据"核实记录　6.3 信息源与版本锚定

目录条目可点击跳转；正文中源码与文档引用挂有永久链接（见 §6.3 版本锚定表）。

0 结论摘要

本报告对题目指定的 7 个工具逐一做了存在性核实与文档级调研，并对每层一个代表——库层 LangGraph、Harness 层 Codex CLI、产品层 OpenClaw——完成源码深读（其余工具对照简述，见 §3.4）。三条核心发现：

发现一：七者分三层，层间是上下游关系而非竞品关系。库/SDK（LangChain/LangGraph、Pydantic-AI）回答"用什么造 Agent"，Runtime/Harness（Codex CLI、dsh、Pi）回答"用什么跑 Agent"，成品/产品（OpenClaw、Hermes-Agent）回答"直接用哪个助手"。跨层直比会误导选型，公平比较方法是分层各回答一个问题（归层依据与关系图见 §2）。

发现二：能力开箱程度随层级上升，定制空间随之收窄。产品层开箱即是能用的助手（长期记忆、20+ 消息渠道），harness 层把执行隔离与审批做得最完整（Codex 沙箱内核级三平台），库层可控性最强、但沙箱与审批须完全自建（长期记忆仅 LangGraph Store 内置）。这是分工而非差距（七维度深析见 §3）。

发现三：企业落地的门槛不在许可证与模型接入，而在治理配套。七者许可证均为 MIT/Apache-2.0，国产模型接入均有原生路径；真正普遍缺失的是权限治理与审计的产品化——OpenClaw 官方明示单一信任域，Codex 审批档位不等同企业 RBAC，金融级落地都须在工具之外补齐治理层（五维度评估见 §4）。

七工具总览（口径：长期记忆按四态标注见 §3，MCP/ACP 按角色标注、方向明细见 §2.2）：

工具	分层	状态管理	长期记忆	HITL	沙箱	MCP	ACP
LangChain LangGraph	SDK	Checkpoint	Store（跨会话）	interrupt / Command	自建	Client	—
Pydantic-AI	SDK	会话存储	未实现（配套包不计入）	Deferred Tools	自建	Client（+FastMCP 打包 Server）	—
dsh	Harness	事件溯源日志	未内置（仅 MCP 参考）	双旋钮审批	原生三平台（Win 不完整）	Client	ACP Server
Codex CLI	Harness	Rollout JSONL	已实现、默认关闭	两轴 + granular	内核级三平台（同层最强）	Client（Server 已移除）	—
Pi	Harness	JSONL 会话树	未实现（哲学：自建/装包）	扩展实现（无内置）	外部容器	Client（内置客户端可换）	—
OpenClaw	产品	SQLite 会话库	较完整（五层+溯源）	渠道内审批	可选（默认关）	Client + Server	ACP Host
Hermes	产品	SQLite + WAL	较完整（文件式+FTS5）	三档 + 硬底线	多后端（local→云沙箱）	Client	ACP Server

Checkpoint、Store 和 interrupt/Command 为 LangGraph 的能力；LangChain 的 create_agent 基于 LangGraph Runtime 实现，两者的抽象层级与职责不同。

四场景推荐结论（论证见 §5）：(a) 金融机构内网合规问答 → 库层自建 RAG + 内网模型；(b) C 端高并发客服 → 库层做无状态服务；(c) 研发编码运维助手 → Codex CLI 为当前生产工具（配企业策略管理）、dsh 为国产化 Runtime 的 PoC / 二次开发候选；(d) 长期记忆助理 → 国内生态 Hermes-Agent、国际渠道 OpenClaw 二选一。题目 2（招股书问答 Agent）选型：Pydantic-AI + 自建确定性层 + 国产模型实测选型（论证见 §5.5）。

1 存在性核实

7 个指代均经 GitHub API + 官网 + npm/PyPI 注册表双通道核实：

工具	核实指代（org/repo）	存在性	最新 release	许可证
LangChain	`langchain-ai/langchain`	✅ 存在	多包分别发版（列 core 1.7.4）	MIT
LangGraph	`langchain-ai/langgraph`	✅ 存在	1.2.19	MIT
Pydantic-AI	`pydantic/pydantic-ai`	✅ 存在	v2.65.0	MIT
OpenClaw	`openclaw/openclaw`	✅ 存在	v2026.10.3	MIT
Hermes-Agent	`NousResearch/hermes-agent`	✅ 存在	v0.22.6（11-02）	MIT
DeepSeek Harness	`deepseek-ai/deepseek-harness`	✅ 存在	dsh-v0.2.2-alpha.3（11-03）	MIT
Codex CLI	`openai/codex`	✅ 存在	rust-v0.166.0	Apache-2.0
Pi	`earendil-works/pi`	✅ 存在	v1.5.0（11-02）	MIT

口径声明：三个深读代表使用当日 main 分支并锚定 commit：`LangGraph 3c916d7`、`Codex CLI bfdbf01`、`OpenClaw 4285c6ce`（2026-11-03）。正文源码引用给出相对仓库根的文件路径，行号与完整清单见随附调研

卡；关键论断处挂源码永久链接（permalink），全部版本锚定见 § 6.3。

2 分层定位与关系图

2.1 三层定位表与归层依据

本题采用题目建议的三层划分，判层标准是**交付物形态**：库/SDK 是 `pip/npm install`、进程内嵌入的代码；Runtime/Harness 是可运行的 agent 执行框架（CLI/守护进程），自带循环、工具、审批但通常不面向最终用户；成品/产品是终端用户直接使用的助手（聊天渠道、桌面/移动端）。归层依据如下：

工具	层级	归层依据（形态与代码证据）
LangChain / LangGraph	库/SDK	安装即库、无守护进程无产品 UI。LangChain 是较高层框架（标准 Agent Loop + Middleware，`create_agent` 基于 LangGraph Runtime），LangGraph 可独立使用、提供底层有状态执行与编排（状态/持久化/恢复/中断）。
Pydantic-AI	库/SDK	纯库进程内运行、无强制服务端；持久化 schema 留给用户。**本题仅评核心库** `pydantic-ai(-slim)`，配套包与商业平台 Logfire 不计入。
DeepSeek Harness (dsh)	Runtime/Harness	不自带开箱业务 Agent，提供会话、agent-loop、工具注册表、沙箱、审批等运行时底座；`dsh` 按 profile 启动 web / headless / sdk / acp 四形态。
Codex CLI	Runtime/Harness	自带 TUI、工具循环、沙箱、审批、会话持久化——是"跑 agent 的壳"，非嵌入库、非托管服务。
Pi	Runtime/Harness	monorepo 分层：pi-ai / pi-agent-core 是库，pi-coding-agent 是 CLI，pi-durable 是 durable 执行 harness；默认交付物是 CLI harness，故归 harness 层。
OpenClaw	成品/产品	20+ 聊天渠道 + 桌面/移动客户端的个人助理形态；交付物与默认用法是产品，故归产品层。
Hermes-Agent	成品/产品	交付物是装好即用的助手：CLI/TUI + Desktop + Web dashboard + 20 余消息平台机器人（含微信/企微/钉钉/飞书/QQ）。

2.2 关系图：嵌入、后端/模型提供、协议互通

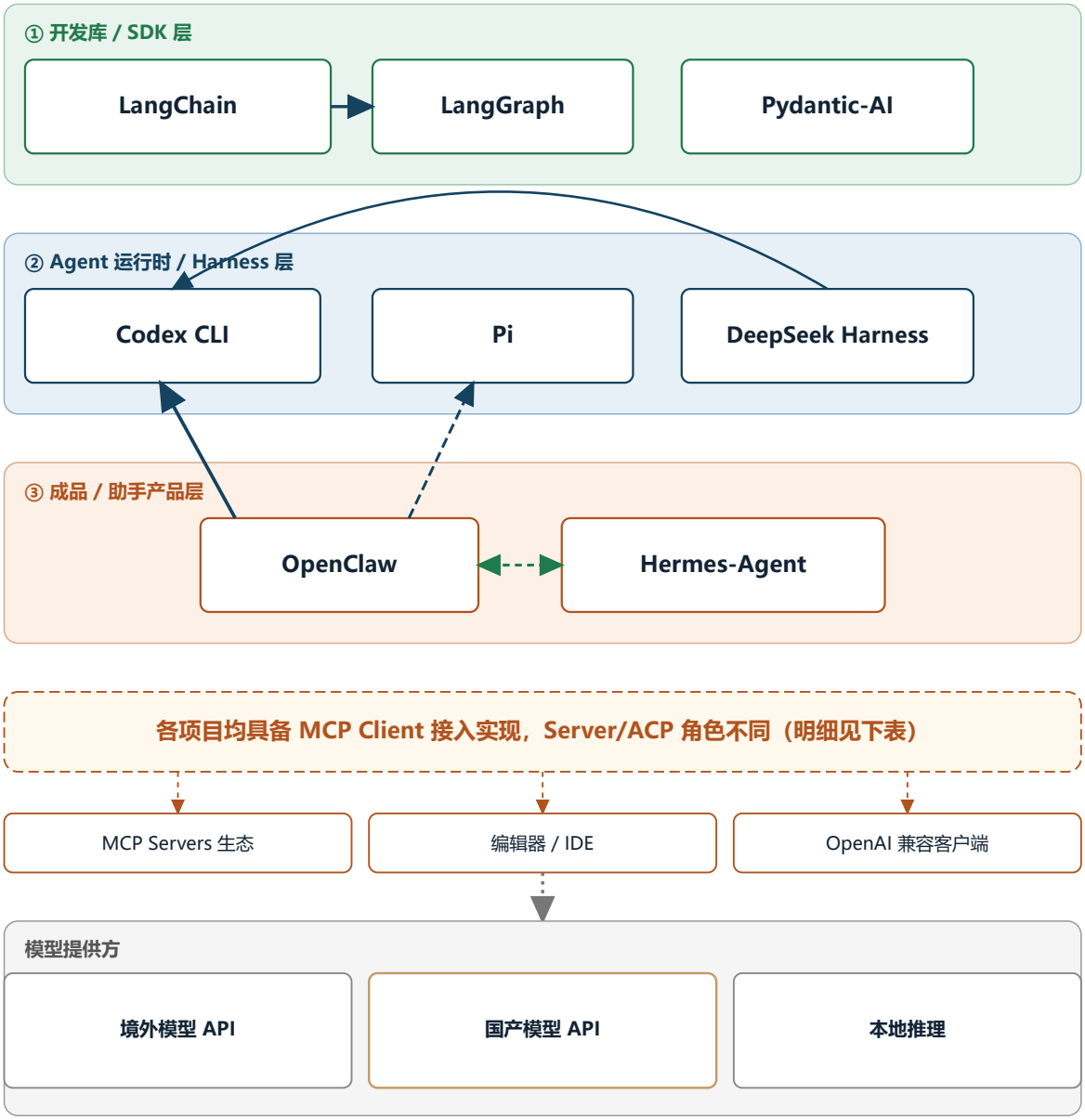


图 1：七个工具的分层与关系。实线深蓝=原生 Runtime 插件（OpenClaw×Codex）；虚线深蓝=外部 ACP 接入（OpenClaw×Pi）；虚线橙=协议互通；双向虚线绿=同层竞品；点线灰=模型提供关系。

- **蓝色实/虚线（嵌入）**：LangChain 的 `create_agent` 基于 LangGraph Runtime；产品层 → Codex（实线）＝实现 OpenClaw harness 契约的原生插件，产品层 → Pi（虚线）＝经 acpx 以 ACP 启动外部 Agent；harness 层内弧线＝dsh 可把 Codex CLI 当 subagent provider。
- **绿/橙/灰线**：绿色双向虚线＝OpenClaw 与 Hermes-Agent 同层竞品（Hermes 提供迁移器）；橙色虚线＝协议互通总线（方向明细见下表）；灰色点线＝模型提供关系，harness/产品层模型无关。

MCP / ACP 方向明细——两个协议的角色不可混用：MCP 的 Client 消费外部 Server 的工具、Server 把自身能力暴露给外部 Agent；ACP 的语义是**客户端启动并管理 Agent**——Agent Server 向 IDE 等客户端提供 Agent 服务，Host/Client 反向启动外部 Agent。七个项目的实际实现方向：

项目	MCP Client（接入外部 Server）	MCP Server（对外暴露）	ACP 角色
LangChain/LangGraph	✅ `langchain[mcp]`（MCP 工具转 LangChain 工具）	—	—
Pydantic-AI	✅ stdio/HTTP + FastMCPToolset	✅ 以 FastMCP 把 agent 打包为 MCP 工具	—
dsh	✅ opt-in（`dsh-mcp-client`，子进程剥离凭据）	—	**Agent Server**：`--profile acp` 供 Zed 等 IDE 调用
Codex CLI	✅ `[mcp_servers]` 配置	✗ 曾提供，2026-09 官方文档确认移除	—
Pi	✅ 内置自研客户端（四档 exposure 治理）	—	—
OpenClaw	✅ `mcp add/doctor/probe`	✅ `mcp serve` + Gateway 对外 OpenAI 兼容 API	**Host/Client**：acpx 插件接入外部 coding harness
Hermes-Agent	✅ `hermes mcp add`	—	**Agent Server**：VS Code / Zed / JetBrains 连接 Hermes

七者的真实关系是可组合的依赖图，而非七个并列选项，§5 选型建议即按"组合方案"给出。

2.3 为什么跨层直比会产生误导，以及本报告的比较方法

误导一：把开发框架的灵活性，当成运行时的能力不足。同做一编程任务：Codex 开箱即用，LangGraph 需先实现工具、状态与任务循环——直接比完成率会得出"Codex 更强"，但 LangGraph 提供的是**构建 Agent 的能力**，Codex 提供的是**已经构建好的 Agent**。

误导二：把 Harness 的默认能力，当成底层模型的能力。Codex 编码表现最好不等于其模型推理最强——Harness 的上下文组装、工具设计与反馈循环同样影响结果，比较必须控制模型变量。

误导三：把成品助手的产品能力，当成 Harness 的执行能力。OpenClaw 的渠道、记忆与定时体验不代表其单次推理更强（它可把其他 Harness 作为执行后端）；Codex 改代码更出色，也不代表它是更好的个人助手产品。

误导四：忽略不同项目替用户承担的工程工作。库层**构建导向**、harness 层**执行导向**、产品层**应用导向**——比较时还要问它已经替你实现了什么、还需补齐什么：**只想完成任务用产品层，要开发自己的应用用 Harness 层，要从零构建自己的 Agent 用库层。**

本报告的公平比较方法：分层各回答一个问题——SDK 层谁更适合用来构建自己的 Agent？Harness 层谁的执行机制更适合自己的任务与运行环境？产品层谁能以最低的集成和运维成本持续完成目标用户的任务？统一以"七维度 × 谁负责"坐标系发问（内置/生态/留给用户三档），以组合方案而非孤立工具为比较单位（§5 四场景均按组合给方案）。

3 架构与核心抽象（深析 ×3 + 简述 ×5）

深读代表：**库层 LangGraph**、**Harness 层 Codex CLI**、**产品层 OpenClaw**。其余五工具简述见 §3.4。能力状态四态：**已实现**／**默认启用**／**实验性・可选**／**未实现**——已实现不等于默认获得完整体验，两者分开表述。

七维度统一口径（统一定义见随附调研卡）：状态／记忆／工具调用／扩展机制（Plugins・Skills・Subagents・MCP・ACP 五类子概念不视为同类扩展）／HITL／可观测性／部署与许可。表述约定：每个维度按**机制 → 证据 → 判断**三句式分行展开；仅对三个最有区分度的机制深读——LangGraph 的 checkpoint（§3.1①）、Codex 的沙箱与管理员策略（§3.2⑦）、OpenClaw 的信任域与供应链（§3.3⑦及专题）。

3.1 LangGraph（库/SDK 层代表）：图执行 + 持久化 checkpoint

① 状态：✅ 有，checkpoint 是 LangGraph 第一卖点

判断：完整实现。双持久化抽象：Checkpointer 管 thread 内图状态快照，Store 管跨 thread 应用数据（见②）；断点恢复、回放、分叉都基于 checkpoint。

- **存储可插拔：**`BaseCheckpointSaver` 定义统一契约，参考实现从 InMemory 到 Sqlite/Postgres，另有 conformance 一致性测试套件——官方有意把存储做成生态
- **设计边界：恢复执行 ≠ 回滚副作用**——checkpoint（含 time-travel 分叉与 interrupt 重放）保证的是**工作流状态**的持久化与恢复，不提供外部副作用的事务性回滚：节点若已写数据库、发过邮件、调过外部接口，恢复到更早 checkpoint 并不撤销这些操作，涉及外部写仍需幂等键、操作日志、补偿事务等工程机制。横向看，Codex 的 rollout 不可变 + revert、OpenClaw 的会话恢复同样只回滚**对话与任务状态**；七者中仅 Hermes 另提供文件级 `/checkpoints + /rollback`（shadow git 快照，opt-in），但也只覆盖本地文件，不触及外部系统

② 记忆：✅ 有（短期 + 长期两套）；语义检索需自备 embedding

机制　短期记忆 = 图状态 channel（`Annotated[X, reducer]` 使多节点并发写同一字段按规则合并而非互相覆盖）；长期记忆 = 独立 BaseStore 数据库，按命名空间 + 键组织，`search()` 支持前缀过滤、结构化 filter 与自然语言 query。
证据　`graph/state.py`、`langgraph/store/base`。
判断　两套接口边界清晰；语义检索不是内建的——库给 `Embeddings` 接口（Postgres 版建 pgvector + HNSW 索引）、embedding 自备。

③ 工具调用：✅ 有，但校验发生在 langchain-core 层

机制　LangGraph 消费 `BaseTool/ToolCall/BaseMessage`，pydantic 校验由 langchain-core 层完成；`ToolNode` 逐 tool_call 执行，工具可返回 `Command` 改状态/跳转；结构化输出双策略（模型原生 / 工具模拟）。
证据　`tool_node.py` 的 `_filter_validation_errors`——LLM 可控参数错误回喂模型自纠、系统注入参数（InjectedState 等）剔除。
判断　校验与自纠闭环完整但不在本库；⚠ `langchain-mcp-adapters` 已停维护、MCP 支持并入主包 `langchain[mcp]`，选型勿引旧包。

④ 扩展机制：⚠ 只有积木，没有编排器

机制　编排原语齐备：subgraph（图本身是 Runnable，子图 checkpoint 与父图共用同一 thread 存储）、`Send` 动态并行（map-reduce）、`Command` 跳转。
证据　`types.py`；supervisor/swarm 编排器在 monorepo 内**未实现**（仅测试 fixture 命中）。

判断 只有积木、没有编排器——官方路径是独立仓 `langgraph-supervisor-py` / `langgraph-swarm-py` + 本库原语；`create_react_agent` 已标 deprecated，能力上移至 langchain 仓的 `create_agent`。

⑤ HITL：✅ 有，且与崩溃恢复共用一套持久化机制

机制 `interrupt(value)`（`types.py`）抛出中断，被 runner 的 `commit()` 捕获，把中断点当作未完成写入存进状态——"等待人工"与"进程崩溃"在存储层是同一种东西；`Command(resume=...)` 时节点从头重放，已答过的审批不重复问；另支持 `response_schema` 类型化审批表单。
证据 `types.py`、`pregel/_runner.py`（permalink 见上）。
判断 恢复语义统一；坑写进文档——节点重跑意味着 interrupt 前代码会再执行（副作用须幂等），不可用 try/except 包 interrupt。

⑥ 可观测性：❌ 原生 OTel 未实现，走 callback 体系自建

机制 观测走 langchain-core callback 体系接自建后端（Langfuse 等有现成集成）；评测集成不内置。
证据 OSS 层无原生 OTel exporter——全仓 grep 仅测试文件命中；主包 deps 不含 LangSmith。
判断 原生 OTel **未实现**；LangSmith 完全可选，不配置即无外发。

⑦ 部署与许可证：✅ 引擎全 MIT，可完全自托管

机制 七包全 MIT（逐一核对 pyproject）；生产最小组合 = `langgraph + langgraph-checkpoint-postgres` + 自写 ASGI 入口；沙箱不内置（库层不直接执行环境操作）。
证据 API server 本体在独立包 `langgraph-api`（`langgraph-cli` 的 `up` import 之）。
判断 自托管完整；供应链注记——langchain-community 包正在停维护收缩（sunset），依赖只走主线包。

3.2 Codex CLI（Runtime/Harness 层代表）：内核级沙箱 + 企业治理面

① 状态：✅ 有（rollout 不可变会话 + resume），但无快照式 checkpoint

机制 会话落盘为 append-only JSONL（真相源）+ SQLite 索引层，fork/回退新开文件、thread_id 稳定；`codex resume / fork / queue` 入口齐全，非交互 `codex exec resume` 保留原 transcript、plan history 与 approvals。
证据 `rollout/src/lib.rs`；用户可感知的"回退"是 Esc-Esc backtrack，非 checkpoint 回放。
判断 有恢复、无快照——代码里的 "checkpoint" 是压缩检查点而非产品级快照，没有 LangGraph 式 time-travel。

② 记忆：✅ 有（AGENTS.md 双层 + memories 两阶段长期管线）

机制 项目记忆（AGENTS.md 沿项目根向上收集拼接）与全局记忆（`~/.codex/AGENTS.md`）双层；长期记忆两阶段管线——后台从闲置 rollout 抽取结构化记忆 → consolidation 合并整编；会话内压缩三路（本地摘要 / 服务端压缩 / token 预算式开新窗）。
证据 `core/src/agents_md.rs`、`core/src/memories/`（工件存 `$CODEX_HOME/memories/`）。
判断 体系完整，但长期记忆**已实现、默认关闭**（`features.memories` 默认 off，需显式开启）。

③ 工具调用：✅ 有（统一 ToolSpec + 两级结构化输出 + MCP 双向）

机制 统一类型 ToolSpec 四型：Function / Namespace（打包省 token）/ ToolSearch（按需加载）/ Freeform——apply_patch 用约束文法让模型直出补丁文本而非 JSON；结构化输出两级（工具结果 output_schema + 回合最终 json_schema(strict)）；MCP 走 stdio + streamable-HTTP 双 transport、白黑名单、per-server 审批。
证据 `tools/src/tool_spec.rs`；校验失败不打断回合——错误分可自纠（回喂重试）与致命（终止）两类。

判断 MCP 为纯 Client 侧接入——曾提供的 MCP Server 模式已于 2026-09 由官方文档确认移除；内置 shell 为可续写伪终端，输出含退出码与耗时。

④ 扩展机制：✅ 有（Skills/Plugins/Hooks/MCP 四通道 + 原生多 agent）

机制 四通道：Skills（`$CODEX_HOME/skills/`，显式 mention 与隐式识别）、Plugins（marketplace：npm 源 / bundle / 远程目录）、Hooks（14 个生命周期事件，可改写命令）、MCP；多 agent 工具组原生（spawn/manage 子代理 + message-board，深度硬上限）。
证据 `hooks/src/lib.rs`；旧 `~/.codex/prompts` 自定义斜杠命令已被 skills 取代。
判断 扩展面收敛清晰、受控——官方明示子代理"只在显式要求时才 spawn，token 消耗高于单代理"；`codex exec --json` 每行一事件的 stdout 契约使其可作 CI/管道编排单元。

⑤ HITL：✅ 有（sandbox × approval 两轴正交）

机制 sandbox 决定"命令能干什么"，approval 决定"越界时问不问"——两轴正交；审批档 UnlessTrusted / OnRequest（默认）/ Granular（实验，五开关逐类）/ Never。
证据 `protocol/src/protocol.rs`；沙箱内被拒后可询问"脱沙箱重试"，但 Never / OnRequest 档下不重试，且策略禁止读的文件同样禁止脱沙箱重试——否则等于悄悄放行。
判断 设计正交、逃逸口管理严谨；exec 无界面模式默认 never，`--dangerously-bypass-approvals-and-sandbox` 与显式审批参数冲突会报错。

⑥ 可观测性：✅ 有（OTel 三信号，隐私默认关）

机制 OTel 三信号可导出（None / Statsig / OtlpGrpc / OtlpHttp）；会话日志三层（rollout JSONL + 文件日志 + SQLite logs 库）；analytics 独立通道默认 POST 到 ChatGPT 后端、`[analytics] enabled=false` 一键关。
证据 `otel/src/config.rs`；指标含 `codex.turn_ttft` / `codex.turn_cost_microusd` 等。
判断 隐私姿态保守——`log_user_prompt` / `log_agent_responses` 默认 false；rollout-trace 为本地调试 bundle，官方明示 "not telemetry… does not upload"。

⑦ 部署与许可证：✅ 有（三平台内核级沙箱 + requirements.toml 企业治理，同类独有）

判断：有。macOS Seatbelt / Linux bubblewrap+seccomp / Windows 三后端；Windows 默认 Disabled 时 workspace-write 自动降级 read-only——宁可降级不裸奔；`requirements.toml` 管理员强制、用户不可削弱。

- **沙箱细节**（`sandboxing/src/manager.rs`）：macOS 网络默认全禁（fail-closed）；Linux 封 io_uring 与 AF_VSOCK 防 WSL2 逃逸；Windows 降级有代码与测试（`config/src/config_toml.rs`）
- **模型接入与治理面**：自定义 provider `[model_providers.<id>]`；⚠ wire_api 只剩 responses（"chat" 已删除并返回迁移报错，`model-provider-info/src/lib.rs`）——第三方网关必须实现 Responses 兼容端点；requirements.toml 白名单（allowed_sandbox_modes / allowed_approval_policies / permission profile 未来新增档默认拒绝 / MCP 身份 allowlist / enforce_residency 驻留锁定）；本地推理走 Ollama / LM Studio
- **认证与密钥**：ChatGPT OAuth（PKCE）或 API key，企业路 Bedrock SigV4 / GCP；auth.json 可迁 OS keyring；provider key 推荐 env_key 或命令式 auth（短时 token）

3.3 OpenClaw（成品/产品层代表）：Gateway 控制平面 + 全插件化 + 供应链防御标杆

① 状态：✅ 有（每 agent 一个 SQLite + 重启恢复三套检测）；time-travel 未实现

机制 每 agent 一个独立 SQLite 库（可换 Postgres），会话记录表只追加 + 序号、配全文检索与向量列；重启恢复三套检测（每轮调模型前先落盘、关机留标记、启动扫孤儿会话）、每会话 3 次恢复预算、crash-loop 熔断。

证据 `openclaw-agent-storage-schema.ts`；明文准则"重启后不重问已批过的授权"。
判断 工程化扎实；checkpoint / time-travel **未实现**——最接近的是保留全部会话记录 + 上下文压缩 + 子代理分叉。

② 记忆：✅ 有（五层体系 + 溯源防投毒 + 混合检索）

机制 五层体系：Instructions（AGENTS.md）→ Curated core（MEMORY.md/USER.md，"做梦"巩固）→ Episodic（日记，只按需召回、永不自动进入对话）→ Prospective（长期意图，时间型编译成 cron）→ Review（DREAMS.md 永不进入对话）；每条记忆带来源标记（本人/agent/不可信/系统，默认按不可信处理），定时任务与子代理产物不能晋升为长期记忆；BM25 + 向量混合检索（中文 CJK 三元组分词），召回双通道（开机按重要性×时间衰减注入 + Active Memory 子代理翻记忆）。
证据 `memory_index_chunks + memory_embedding_cache` 双表；对照 OWASP ASI06 与 MINJA 建模。
判断 写路径即安全边界——网络内容标不可信，防投毒是设计目标而非附注。

③ 工具调用：✅ 有（五档 exec 策略 + 多层准入 + MCP 双向）

机制 shell 执行独立五档策略（deny / allowlist / ask / auto / full，auto = 模型预审→人工兜底，策略钩子 fail-closed、15s 超时变拒绝）；工具准入多层叠加——agent / global / default 三源合并（deny wins）+ 按 provider、按发送者 + 部署方具名 allowlist，MCP 工具再过 sandbox.tools 一道门；MCP 双向（客户端 + 自身暴露成 server + relay 反向中继）。
证据 `zod-schema.agent-runtime.ts`；子代理硬禁名单（`agent-tools.policy.ts`：叶子子代理禁 sessions_spawn / gateway 级工具、禁直接对用户发消息）；`api.registerTool` TypeBox 运行时校验 + 模型坏输出修复层。
判断 防线多层，且子代理权限收敛比主 agent 更严。

④ 扩展机制：✅ 有（harness 是一级插件契约，产品层可整层复用 harness 层）

机制 180+ 个插件 + 约 50 个注册面（manifest 惰性激活 + contracts.tools 契约）；40+ 事件钩子的注入风险 / 超时行为 / 出错放行还是拒绝写进 `PLUGIN_HOOK_METADATA`；harness 是一级插件契约——外部引擎接入需实现四个接口（跑一轮 / 压缩 / 分叉会话 / 认证指纹）。
证据 `src/agents/harness/types.ts`；注册表硬规则：id "openclaw" 保留给内置运行时。
判断 产品层可整层复用 harness 层：现有 harness 按集成方式分两类——原生插件（内置 openclaw[旧名 pi，与 Pi 项目无涉] + **codex app-server（精确 pin `@openai/codex: 0.164.0`，nativeCompaction 独占）** + copilot 等）、外部 ACP 接入（acpx 启动 Claude Code / Gemini CLI 等，Pi 亦经此路径）；子代理 sessions_spawn 隔离分叉，嵌套深度上限 1-5、越深可用工具越少。

⑤ HITL：✅ 有（审批在聊天渠道内以按钮卡呈现，等待期计时暂停）

机制 两阶段审批（`before_tool_call` 返回 requireApproval → 网关注册并等待）；审批交互发生在用户所在聊天渠道内的按钮卡上，无按钮渠道回退 `/approve` 命令（非授权 sender 拒绝）；等待审批期间 run 计时暂停。
证据 拒绝的终局语义直接写给模型——"This denial is final… Do not run the tool call again"（`agent-tools.before-tool-call.approval.ts`）。
判断 渠道内审批体验完整；运行中可直接塞入新指令（steering），提权 exec 是全局明示的逃逸口（tools.elevated 双门）。

⑥ 可观测性：✅ 有（单一诊断漏斗双导出器，隐私默认收敛）

机制 单一诊断事件漏斗接两个导出器（OTel 三信号 + Prometheus 30+ `openclaw_*` 指标，必须走 Gateway 认证）；Gateway 同端口 Dashboard（审批卡 / 日志 / 会话）。
证据 diagnostics-otel 对 provider 调用打 W3C traceparent 标记防伪造调用链。

判断 隐私默认收敛——原始 prompt / 回复 / 工具 IO 一律不导出（要导出须逐项显式开且导出前仍脱敏），日志脱敏恒开、不可配置关闭。

⑦ 部署与许可证：✅ 有（完全自托管）；⚠ 沙箱默认关闭

判断：有，但沙箱要显式开。无付费层 / 无托管服务 / 无 token；容器沙箱体系完整但**默认关闭**（"Tools run on the host unless you configure sandboxing"）——个人助理默认姿态，企业须显式加固；一个 Gateway = 一个信任域（SECURITY.md）。

- **部署形态**：Docker（digest 钉住 / 非 root / cap_drop / no-new-privileges）+ 无公网 ingress 加固模板
- **沙箱与密钥**：Docker 后端 8 项必填、默认镜像无网络、危险目录挂载默认拒绝；SecretRef 三源（env / file / exec——官方示例 1Password op / Vault / SOPS）+ 明文凭据审计扫描
- **数据本地性**：默认全本地，唯一默认外联是每日版本检查（可关）；模型/渠道流量直连服务商、无中继云

供应链防御专题：把"信任"做成版本库里的台账

- **默认冷却**：`pnpm-workspace.yaml` `minimumReleaseAge: 10080`（7 天）+ strict——任何依赖新版本必须"冷"满 7 天才进安装图，直接挡掉"即发即投毒"窗口
- **豁免即台账**：每条豁免带评审依据（PR 号 / 事件）+ 责任人 + remove-after 期限——`@lancedb/lancedb@0.41.0` 标注 "Approved LanceDB compromised-CA remediation"（上游 CA 受陷修复版即时放行）；`@openai/codex` 因信任分级整体豁免——OpenAI 发布链被显式信任到可豁免冷却，其余 npm 生态不可
- **反面记录**：2026 年初爆发期安全事件密集——CCB Belgium 公告 1-click RCE（CVSS 8.8，影响 <2026.1.29）；ClawHub 出现大规模恶意 Skills 供应链事件，**不同研究机构在不同时间窗口识别到数百个恶意 Skills**（如 IMDA 报告引 Koi Security：2026-02-01 发现 341 个、02-16 报告 824 个；Unit 42 对 02—05 月的后续分析仍发现 5 个绕过初期治理的样本——检测命中、确认恶意、已下架、仍可下载是不同口径，不混用单一数字）；多所机构发布停用通告。企业落地必须走 hardened baseline（sandbox all + 权限档 + 审批默认开），并对 Skills 安装实施来源准入、静态扫描、版本锁定与人工审查

3.4 其余五工具简述（相对同层深读代表的关键差异）

七维度全量概览见 §0 总览矩阵；安全、合规、国内可用性等企业维度见 §4 各表。本节只写各工具与同层深读代表（LangGraph / Codex / OpenClaw）的关键差异：

工具	相对同层代表的关键差异	企业/国内结论（详见 §4）
LangChain vs LangGraph	同仓不同位：LangChain 是较高层框架（`create_agent` 标准 Agent Loop + Middleware，基于 LangGraph Runtime）；LangGraph 提供底层图编排与持久化。supervisor/swarm 编排器在 LangChain 侧现成（多 agent 五模式）；结构化输出双策略（模型原生 / 工具模拟）。	中文资料 7 者最厚；DeepSeek / Ollama 官方合作包；Qwen 走第三方包、community 停维护应避开（§4.5）。
Pydantic-AI vs LangGraph	类型优先而非图优先：Agent 无状态，跨 run 状态由 Conversation 显式携带（LangGraph 是 thread 内图状态 + checkpoint）；记忆即消息历史 + BM25 检索，无独立记忆系统；Deferred Tools 把 HITL 做成一等公民、挂起状态随会话持久化；有意不做插件系统，靠 capability 组合扩展。配套包 pydantic-ai-harness 非本题对象。	依赖面 7 者最小；官方明示"审批不是授权边界"、执行隔离自理（§4.1）；v1→v2 API 更名，注意旧教程（§4.4）。

工具	相对同层代表的关键差异	企业/国内结论（详见 §4）
dsh vs Codex	同为日志派会话但强调事件溯源：只追加事件日志 + 格式版本迁移链，状态由重放重建；插件三形态（npm / 配置组合 / 动态插件=agent 自写、带审批门），子代理可纳 Codex 为执行后端；沙箱同为三平台，但 Windows 隔离不完整（Codex Windows 为内核级）；每工具强制结构化输出定义；长期记忆无内置。	治理三缺口（默认日志上传、preview 无审计、无 SSO/RBAC）是金融落地主要障碍（§4.1–4.2）；全量双语文档、可全境内运行；全 prerelease 须锁版本（§4.4）。
Pi vs Codex	极简哲学是 Codex 大而全体系的反面：仅 8 个内置工具（默认开 4 个）；ExtensionAPI 可同名覆盖全部内置工具、接管压缩与信任决策——扩展是灵魂而非附加通道；pi-durable 提供检查点式持久执行（**实验性 • 可选**）；codemode 在 QuickJS 沙箱跑脚本编排批量工具调用（token 经济性设计）；无内置权限系统，靠扩展实现审批。	国产模型原生内置 6 家（§4.5）；供应链自律最强之一；无沙箱、无权限系统，企业必备件全自建（§4.1）。
Hermes-Agent vs OpenClaw	同为产品层但工程重心不同：OpenClaw 以 Gateway 控制平面 + 双插件化为中心，Hermes 以状态工程化见长（SQLite+WAL 多进程仲裁、受监督守护进程、退役日志取证）+ 国内消息平台适配最深（微信/企微/钉钉/飞书/QQ 官方 API 直连）；记忆为文件式冻结快照 + FTS5 全文检索（OpenClaw 为五层 + 向量混合）；审批三档 + 无人值守默认拒绝 + 硬底线黑名单。	红旗：hooks 目录放入即加载无确认门、单用户非多租户（§4.1）；国内整链路最短路径之一（§4.5）。

4 企业落地考量（中国境内金融/大型企业视角）

4.1 安全：沙箱、权限模型、密钥管理、供应链

执行隔离与审批：

工具	执行隔离（沙箱）	权限 / 审批模型
LangChain/LangGraph	无（库层不管；LangSmith Sandboxes 为境外 SaaS）	无内置，代码级自建
Pydantic-AI	无（核心库不管执行隔离）	Deferred Tools 审批（**官方明示"非授权边界"**，端点鉴权自理）
dsh	Linux bwrap / Landlock、macOS Seatbelt、**Windows 隔离不完整**（自报告）；无后端即拒（fail-closed）	沙箱三模式 × 审批两旋钮 + 预设组合；审批默认拒绝（fail-closed）
Codex CLI	**Harness 层最强**：三平台内核级强制（Windows 默认 Disabled 自动降级 read-only）+ 管理员强制策略	两轴正交 + granular 五开关；**requirements.toml 管理员强制为同类独有**——但审批档位**不等同企业 RBAC**
Pi	**无内置**，官方文档化容器化三模式	无内置权限系统；扩展钩子可拦截 + 界面确认；项目信任只管加载不管工具
OpenClaw	**⚠ 单一信任域：一个 Gateway = 一个受信任操作员边界（官方明示），工具审批不是多租户授权边界**——企业安全评价的第一前提；容器沙箱体系完整但**默认关闭**	五档 exec 策略 + 多层工具准入 + 渠道配对身份门 + elevated 提权双门
Hermes-Agent	靠终端后端：local 无隔离（靠审批层）+ Docker 硬化容器 + SSH / 云沙箱	**审批分层最细**：三档 + 无人值守默认拒绝 + 硬底线黑名单（全放行也越不过）

密钥管理与供应链：

工具	密钥管理	供应链风险
LangChain/LangGraph	走 env 约定，应接 KMS / Vault；**checkpoint 静态加密是一等能力**（AES）	依赖面小但包布局变化快（community 收缩 / mcp 并包），须锁版本 + 私有镜像
Pydantic-AI	密钥全 env；MCP 配置 `${VAR}` 展开不入盘	**依赖面 7 者最小**（slim extras 全可选），可控性最佳
dsh	凭据存 `$DSH_HOME/.credentials.yaml`（界面回显脱敏）	SAFETY.md 自认**未做安全审计**；**动态插件是独有攻击面**（agent 自写、有审批门）；注册表冒名包实锤
Codex CLI	file / keyring / auto 三选 + 命令式 auth 短时 token；子进程环境默认滤 KEY/SECRET/TOKEN	cargo-deny 依赖审计；风险 = 周级发版节奏、无插件签名机制
Pi	models.json 支持 `$ENV` 插值与 `!command`（可接外部密钥管理器）	**自律最完整之一**（锁版本 + 冷却期 + OIDC 签名 + SHA256SUMS）；但扩展进程内全权限执行，装前人工审代码是唯一防线
OpenClaw	SecretRef 三源（env / file / exec：1Password / Vault / SOPS）+ 明文凭据审计扫描	**防御体系 7 者最完整**（7 天冷却 + 台账 + GHSA 规则化）；但有真实事件史：CCB RCE + ClawHub 恶意 Skills 供应链事件（见 §3.3 口径注记）

工具	密钥管理	供应链风险
Hermes-Agent	凭据保险库（Bitwarden / 1Password）；MCP 子进程 env 白名单	hooks 目录"放入即加载"无确认门（文档自认），须纳入基线审计

安全面的三个跨层结论：沙箱强度（同层比较）Codex（内核级、三平台、默认降级保护）＞ dsh（三平台但 Windows 隔离不完整且无审计）＞ Pi（无，靠外置容器）——库层与产品层不参与此排序（前者不执行、后者的威胁模型在网关与渠道侧）；产品层的核心安全命题不是沙箱而是**渠道攻击面**——OpenClaw 事件史（§ 3.3）证明"常驻网关+聊天机器人"的暴露面与传统 Web 服务同构，须按互联网服务标准加固与披露跟踪；供应链正成为分层新维度——OpenClaw（冷却+台账）与 Pi（pin+release-age）把依赖治理做进工程默认，dsh/Hermes 的插件/技能市场面需企业自有准入控制兜底。

4.2 合规：私有化、数据不出境、审计留痕、许可证

工具	私有化部署	数据不出境能力（关键差异）	审计留痕
LangChain/LangGraph	纯库内嵌或 Agent Server 自托管 docker	✅ 框架侧零强制境外依赖（LangSmith 可选不装即无外发）；checkpoint 默认**明文存全量对话**，须强制 AES+库权限+保留策略	callback 体系接自建后端；无原生 OTel exporter
Pydantic-AI	纯库自托管，无云依赖	✅ 框架零强制外发：模型端点全自定（内网 vLLM）+ Logfire 不装即无外发（装了可 `include_content=False` 或改发自建 OTLP）	原生 OTel 三信号，接国产 OTLP 后端
dsh	dsh web 默认只监听本机	⚠ **重大红线：默认启用 dsh-session-log-deepseek——用官方 API 时每请求附带最高 8 MiB 增量会话日志上传 DeepSeek**（境内方向但金融必须显式关闭）；遥测可彻底关闭	审批记录落持久日志；但**无集中审计管道、无 SSO/RBAC**
Codex CLI	本地运行无强制云组件	⚠ 默认模式模型流量+OTel 指标出境至 OpenAI；API-key+境内端点模式日常流量不经 OpenAI；`enforce_residency` 仅锁 us	rollout JSONL 会话真相源 + SQLite logs + OTel 可指自建端点（支持 mTLS）
Pi	纯本地（模型目录缓存离线可用）	✅ 运行时唯一外联=模型端点（可全国内/本地）；/share 与 /bug 上传需禁用	无内建 OTel 导出；`tool_execution_end` 扩展钩子是自建审计日志的正确挂点
OpenClaw	完全自托管；docker 多形态，含无公网 ingress 加固模板	✅ 数据默认全本地（~/.openclaw），对基金会唯一默认外联=每日版本检查（可关）；模型/渠道流量直连服务商无中继云	OTel/Prometheus 双导出 + 日志 redaction 恒开；无 SIEM 级集中审计产品模块
Hermes-Agent	自托管完整（含 Docker 沙箱镜像）；Hermes Cloud 在境外云，境内只走自托管	✅ 所有状态本地 ~/.hermes；**消息网关直连平台官方 API、不经 Nous 服务器**；OTLP 无内容导出	会话全量落 SQLite+FTS5 可检索；OTLP 可接自建 Collector；无 RBAC 产品模块

口径说明：表中"外发"指框架自身默认行为——运行时是否出境还取决于模型端点、遥测/日志与第三方工具配置（配内网端点 + 关闭可选遥测即可全境内）。**许可证横向**：7 者全部 MIT 或 Apache-2.0（Codex），商用无限制。合规分水岭不在许可证而在**数据流向的默认值**——框架默认零外发：Pydantic-AI / Pi / LangGraph；

数据本地、渠道与 provider 流量直连境外服务商（可配国产端点）：OpenClaw / Hermes；默认出境（可改）：Codex；有一个必须显式关闭的默认上传项：dsh。

4.3 成本：token 开销放大因素与运维成本

token 放大的主要机制与对策（跨工具通用，按可控性排序）：

- **多 agent 扩散与上下文压缩**：Codex 官方明示子代理"token 消耗高于单代理"且默认只在显式要求时 spawn；压缩策略各家齐备——Codex 三路（本地摘要/服务端/预算式开新窗）、Hermes 有损摘要、Pi 结构化摘要 + **codemode**（模型写脚本编排批量工具调用，只有脚本输出回模型）、dsh 工具定义白名单投影
- **记忆与状态注入**：Hermes 记忆冻结快照、OpenClaw 只注入 curated 层（日记层永不自动注入）——省 token 与安全兼顾；LangGraph 落盘档位影响的是数据库写放大而非 token；SQLite 会话库需保留策略（OpenClaw 默认 30 天自动清理）
- **Prompt Cache 命中率（隐性放大器）**：系统提示词、工具 Schema、Skills 与长期记忆注入都进入输入前缀——**前缀稳定才能命中缓存，命中与未命中价差可达数倍**，动态注入会破坏前缀导致全价重算。佐证：Hermes 冻结快照专为保前缀、Pi models.json 声明 promptCache、Pydantic-AI 遥测内置 hit_ratio 告警——评估成本时应把缓存命中率与上下文压缩同列为首要观测项

运维成本按层分三档：库层无额外守护进程，但需自建服务架构、状态存储与模型网关等基础设施——主要成本是**工程建设与平台运维**；Harness 层是"每开发者一份"的终端工具，无服务端运维（dsh web 模式例外）；产品层运维常驻守护进程（进程监督 / SQLite 维护 / 渠道凭据轮换 / 版本更新列车）。**版本跟进成本是隐藏大头**：Codex 周级、Hermes 日更量级、dsh 全 prerelease 且明示破坏性变更、Pi 小版本也可能破坏兼容——企业全部需要"锁版本 + 升级窗口管理"，dsh 是 7 者中版本风险最高者。

4.4 维护成熟度

工具	项目年龄	治理与核心维护者	活跃度	成熟度判断
LangChain/LangGraph	2022 年初（最老）	LangChain Inc. 商业公司，生态最成熟	14.9 万/4.4 万★，当日 push	**生产级**；大版本重构期已过（v1.0），文档质量高
Pydantic-AI	2024 末	Pydantic 团队（类型生态口碑背书）	2.1 万★，当日 push	**生产级**；Harness 包 0.x 另计
dsh	2026-08（3 个月）	DeepSeek 官方 org	27.1 万★（3 个月现象级），当日 push，全系列 prerelease	**developer preview**：架构成熟度极高（事件溯源+格式版本化纪律）但产品/企业就绪度低（无审计、无 SSO）
Codex CLI	2025-04 开源	OpenAI 官方，专职团队	13.3 万★，当日 push，周级发版	**生产级**（但迭代过快，企业须锁版本）
Pi	2025-08	Mario Zechner 主导 + mitsuhiko（Armin Ronacher）；Earendil Works PBC	12.0 万★，当日 push，npm 周下载百万级	**1.0 已过**（v1.5）；社区维护、issue 门槛模式对紧急求助不友好

工具	项目年龄	治理与核心维护者	活跃度	成熟度判断
OpenClaw	2025-11（Clawdbot）	**独立基金会 501(c)(3) 非营利**：多家机构捐赠、与 30 余家组织合作	39.3 万★，近期常态 push，10 月内连发安全补丁列车	**治理成熟 ≠ 产品安全成熟**：基金会治理与披露文化成熟（600+ 条公告），但安全事件史说明产品攻击面曾被验证——两项分开评价，生产落地须走加固基线
Hermes-Agent	2025-07	Nous Research（teknium1 联创主导），数百 contributors	26.0 万★，当日提交	**生产级倾向**，但日更量级=回归风险并存

过程指标之外，成熟度更要看质量指标——stars、commit 与测试数量是活跃度信号，不直接证明生产成熟度：核心维护者集中度（Bus Factor）Hermes 联创主导、Pi 双主力——集中意味着路线连贯但单点风险，dsh 六人千级 commits、OpenClaw 基金会群体相对分散；Issue/PR 响应与发布质量——dsh 不收 GitHub issues、Pi 对新贡献者 issue/PR 默认自动关闭（求助有门槛）、LangGraph 单 release 窗口合并数百 PR，CI 质量 Pi（OIDC+SHA256SUMS）、OpenClaw（provenance 签名+600+ 安全披露）、dsh（per-file 覆盖门+双语文档门禁）、Codex（cargo-deny+Bazel）；Breaking Change 频率见 §4.3。

4.5 国内可用性：国产模型、断网运行、中文资料

工具	国产模型接入	无境外网络依赖运行	中文资料
LangChain / LangGraph	DeepSeek/Ollama 官方合作包；Qwen 走第三方、智谱走 community 包	✅ 框架零强制境外依赖；LangSmith 可选	★★★★★ 官方生态外中文教程最厚
Pydantic-AI	DeepSeek/通义/月之暗面官方 Provider；智谱走兼容端点	✅ 可全境内（模型端点全自定）	★★☆ 无官方中文，社区镜像停在 v1 时代
dsh	内置 DeepSeek/GLM/Kimi + 任意兼容端点三协议	✅ anthropic/openai 可完全不配；遥测可全关	★★★★★ **唯一全量中英双语文档**（.zh.md 成对）
Codex CLI	**DeepSeek 官方一级支持**（中文文档+一键脚本+原生 Responses API）	⚠️ API-key+境内端点可行；安装/OAuth 默认域名需代理	★★★★ DeepSeek 官方中文文档+知乎/腾讯云多篇
Pi	**原生内置最全**：六家国产厂商带专属 env key（多家含 China 专属端点）	✅ 运行时唯一外联=模型端点；pi.dev/GitHub/npm 在境外国（镜像解决）	★★★ 无官方中文；runoob/知乎/B 站已有规模
OpenClaw	内置 provider 覆盖最广：国内主流厂商全家 + Ollama/vLLM/SGLang 本地栈	✅ 模型+渠道（飞书/QQ/微信）全国产可组合；ClawHub 插件源可强制改用 npm 源规避	★★★★ **官方文档有 zh-CN 版**；飞书官方插件
Hermes-Agent	五家原生 provider，多家自动探测大陆/CN 端点	✅ 模型+消息平台全境内可；安装/更新/Skills Hub 链路依赖境外源	★★★★ 官方 README 中文版+CLI i18n

国内可用性总评：文档维度 dsh（全量双语）与 OpenClaw（官方 zh-CN + 飞书官方生态）领跑；模型接入 Pi（六家原生含 China 专属端点）、Hermes、OpenClaw 最全，Codex 有 DeepSeek 官方一级支持的特殊优势；整链路境内 Hermes（国产模型 + 国内消息平台 + 自托管网关）与 dsh 是最短路径，但两者的企业治理缺口（见 §4.1–4.2）要求落地时自建配套。

5 场景选型建议

5.1 场景 (a)：金融机构内网的合规问答

需求画像：私有化部署、数据严格不出境、答案可溯源、审计留痕、并发规模有限。

> **推荐：Pydantic-AI（首选）或 LangChain/LangGraph：库层自建 RAG 问答 + 内网模型端点**
>
> **理由**：合规问答是**确定性工作流 + 检索**，不需要自主执行 shell/文件的 agent 能力——工具执行面就是攻击面，本场景应**最小化工具集**（只读检索工具）——库层甜区：行为可控、每步可审计、对话数据落哪家库带哪个租户列全由企业自定（Pydantic-AI 依赖面最小、SBOM 审计友好，结构化输出 `output_type` 强制引用字段 schema、校验失败自动回喂）。若流程含多步审批（答案需合规官复核后发布），LangGraph 的 interrupt() / Command(resume) 持久化审批语义现成。
>
> **主要风险**：检索质量是答案质量的天花板（合规文档多口径表述需检索层显式处理）；LangGraph 备选时 checkpoint 默认明文存全量对话，须强制 AES + 库权限 + 保留策略；引入执行类工具须自配隔离（库层无沙箱）。

否决：OpenClaw / Hermes 多租户隔离不足的个人助手架构，20+ 渠道与工具执行面是多余攻击面（§ 4.1，OpenClaw 另有 RCE 事件史 § 3.3）；dsh preview 状态 + 审计缺口 + 默认日志上传（§ 4.1–4.2）；Codex 定位错配且默认流量出境；Pi 无权限系统、企业必备件全自建——对本场景"自建"的部分，库层反而更轻。

5.2 场景 (b)：面向 C 端的高并发客服

需求画像：水平扩展、低延迟、成本敏感、结构化输出（工单/意图分类）、故障隔离、内容安全。

> **推荐：Pydantic-AI 或 LangGraph：服务端框架层做无状态服务**
>
> **理由**：两者都是库，扩展性 = 你的服务架构的扩展性——请求与资源生命周期由服务方完全掌控；Pydantic-AI 路线 Agent 全局无状态、`UsageLimits` 限制单次运行的请求数/工具调用/Token 与估算成本（成本限制依赖模型计量信息，跨请求与租户预算仍需外部系统管理）、FallbackModel 多模型容灾；LangGraph 路线 checkpointer 挂 Postgres、thread_id 即会话句柄。须写明：**它们自身不是现成的多租户 Web 服务**——HTTP 服务、任务队列、会话隔离、限流熔断仍需开发团队补齐。
>
> **主要风险**：结构化输出保证格式、不保证事实——C 端幻觉与内容安全需检索接地 + 审核层兜底；Pydantic-AI v1→v2 有 API 更名、LangChain 处于包布局震荡期，生产须锁精确版本；LangGraph 会话数据全量落 Postgres，需加密与保留策略管控。

否决：dsh 虽具备服务化形态（web / headless / sdk / acp 四 profile）但 preview 状态 + 单用户鉴权 + 多租户治理缺口（§ 4.1–4.2）；Codex 定位与执行抽象偏编码任务；Pi 服务层、隔离与权限系统全部自建；OpenClaw / Hermes 多租户隔离不足（§ 4.1）——判断口径是"是否建议直接用作生产级 C 端客服平台"，不是"能否用于该场景"（任何工具作底层 Runtime 二次开发，需按 § 2.3 误导四单独验证）。

5.3 场景 (c)：研发团队内部的编码与运维助手

需求画像：真实编码能力、命令执行、沙箱隔离、审批、企业统一治理、国产模型可接。

> **推荐：Codex CLI 为当前优先考虑的生产工具；dsh 为国产化 Runtime 的优先 PoC / 二次开发候选；OpenClaw 作消息化入口（可选）**
>
> **理由**：Codex **沙箱为 harness 层最强**（三平台内核级强制）+ **requirements.toml 是 7 者中唯一的管理员强制配置体系**，天然适配企业统一管控（§4.1）；DeepSeek 官方一级支持，日常推理流量可不经 OpenAI（§4.5）。dsh 第一方 + 全量双语文档 + 任意兼容端点三协议，但 preview 状态 + 审计能力 + 默认日志上传三重治理缺口（§4.1–4.2），**当前证据不足以支持其作为 Codex 的生产替代**——定位为国产化 Runtime 的优先 PoC，试点中必须显式关闭 session-log 上传；**国产模型可接入 ≠ Harness 已适合生产落地**。可选叠加 OpenClaw：把 Codex 挂为 harness 插件、经飞书/钉钉渠道给团队提供消息化入口（飞书官方插件，生产可用）。
>
> **主要风险**：两者上生产前均应经过**内部安全评审、任务成功率与模型兼容性测试**——本报告证据可支撑"Codex 优先考虑、dsh 优先 PoC"的排序，不足以替代企业内部验证；Codex `wire_api` 锁死 Responses 协议——第三方网关必须实现 Responses 兼容端点（国产模型中 DeepSeek 原生支持）；周级发版要求企业锁版本 + 跟踪 changelog；OpenClaw 叠加层必须走加固基线（沙箱全开 + 审批不默认关闭）并跟踪其安全披露。

否决：LangChain/LangGraph、Pydantic-AI 库层要从零造 harness——TUI、会话、沙箱、审批全自建，对"给研发团队一个能用工具"的目标是重复发明；Pi 无内置权限系统/沙箱，企业必备件全自建——适合有 TS 平台工程团队当底座二次开发，不适合直接下发；Hermes 个人助手形态，沙箱 / 治理 / IDE 集成不如 Codex 专精。

5.4 场景 (d)：需要长期记忆的个人或团队助理

需求画像：跨会话记忆质量、多渠道触达、主动性、7×24 无人值守、隐私可控。

> **推荐：国内平台生态 → Hermes-Agent；国际渠道 → OpenClaw（同层直接竞品，二选一）**
>
> **理由**：Hermes（国内首选）记忆工程化扎实（冻结快照保前缀缓存 + 跨会话全文检索）+ **国内消息平台适配 7 者最深**（微信/企微/钉钉/飞书/QQ 官方 API 直连、免公网端点）+ 状态持久化工程化罕见（SQLite+WAL、跨整机重启自动续接会话）+ HITL 三档硬底线适合无人值守。OpenClaw（国际渠道首选）记忆体系文档层最完整（五层 + 防投毒溯源 + 混合检索 + 常驻意图）+ 模型与 harness 双插件化 + 治理与披露文化成熟。选择轴：渠道生态 + 安全偏好（审批分层→Hermes；供应链防御→OpenClaw）。
>
> **主要风险**：两者都是"个人/小团队可信操作员"设计——多用户共享需隔离自建（非多租户）；OpenClaw 事件史要求加固与版本跟踪；Hermes 日更量级要锁版本；微信个人号群聊受限。
>
> **核心判断标准：记忆系统最重要的不是能存多少，而是能否在正确的时间把正确的信息提供给正确的用户。**机制不直接等于记忆效果——公开的记忆质量评测数据不存在，落地前应按记忆七指标自建评测（两工具设计机制对照见调研卡）。**团队场景注意 Memory Isolation**：两者默认以 agent 为记忆边界——团队共享一个助理即共享全部记忆，可行解是按成员拆分 agent 或显式接受团队级共享。

否决：Codex memories 长期记忆默认关闭（§3.2）、无消息渠道——编码工具不是团队助理；Pi 无内建长期记忆（极简哲学要求记忆自建或装包）；dsh 长期记忆无内置（仅三个默认关闭的第三方 MCP 参考）；库层两者记忆、渠道、守护进程全要自建，达到产品层体验的工程量不经济。

5.5 题目 2：招股书可溯源问答 Agent 的工具组合

题目 2 的核心是面向固定金融数据集的可溯源问答，涉及 PDF 文档与结构化数据库两类数据源，要求支持信息检索、SQL 查询、数值计算、原文引用、拒答和严格结构化输出。其主要挑战在于数据处理、证据检索与

结果校验，而非 Agent 自主规划和执行复杂任务。

> **推荐组合：Pydantic-AI + 自建数据处理与检索层 + 确定性工具**
>
> - **Agent 框架**：Pydantic-AI——作为轻量 SDK 嵌入批处理入口，负责问题理解、工具调用、任务路由和结构化输出；其类型约束与输出校验机制适合本题严格的 JSON Schema 要求
> - **数据处理与检索**：独立实现——PDF 解析、表格抽取、跨页合并、页码映射和混合检索属于数据工程能力，不应依赖 Agent 框架完成；检索层负责返回带文档 ID、页码和原文片段的证据
> - **工具层**：确定性实现——SQL 查询、单位换算、增长率计算和证据校验通过独立工具完成，避免将数据库安全与数值准确性寄托于 LLM
> - **模型层**：可替换的国产模型 API——根据工具调用能力、结构化输出稳定性、中文效果和成本进行实测选择
>
> **关键边界**：Pydantic-AI 通过 output_type 和类型校验机制约束输出结构，但不能保证引用真实、数值正确、证据完整——这些能力必须由检索、原文定位、确定性计算和校验组件提供，这正是 Agent 框架与业务工程组件的职责划分。

其他工具的适配性：

- **LangChain / LangGraph** 是可行备选，生态集成和复杂流程编排能力更丰富；若后续增加多阶段状态管理、持久化恢复或人工审批，可考虑引入 LangGraph
- **Codex CLI、Pi 和 dsh** 更偏向 Agent 执行与编码任务场景，虽然具备不同程度的非交互运行能力，但不是本题轻量问答服务的首选
- **OpenClaw 和 Hermes-Agent** 面向持续运行的助手产品，其消息渠道、长期记忆和审批机制超出本题必要范围

主要风险：

- 框架提供 Schema 校验能力，但不能保证事实正确和引用真实——必须由检索与校验层保障
- 国产模型的工具调用和结构化输出能力存在差异——需要通过题目集实测选择

结论：本题应优先选择库 / SDK 层，以 Pydantic-AI 承担 Agent 控制与结构化交互，数据解析、检索、SQL 和计算由独立确定性组件完成。**核心原则是让 LLM 负责理解与决策，让确定性程序负责数据处理、计算和证据校验。**

6 附录：调研方法与信息来源

6.1 调研方法

五阶段流水线：存在性核实 → 文档级调研卡 → 源码深读（三代表仓库克隆逐文件阅读，§3 源码引用出自深读笔记 `research/*-deep.md`，完整文件清单、行号与统计数字见各调研卡）→ 综合写作 → 引用抽查终检。**反幻觉纪律**：调研中与任务书预设冲突的结论（如"FastMCP 已捐给 Pydantic"被证伪、Codex 多处流传口径被源码推翻）均以查证证据为准修正。

6.2 "查无实据"核实记录

本报告**未发生**"工具查无此物"的情形。如实标注的"未找到"项：LangGraph OSS 层原生 OTel exporter；OpenClaw checkpoint / time-travel 专门功能与产品内评测框架；Pi 内建长期记忆；Pydantic-AI 智谱内置 provider 与 vLLM 官方 provider 页；dsh 独立分布式 trace 视图与产品级评测数据集——详情见调研卡。

6.3 信息源与版本锚定

源码深读工具（固定 commit SHA）——正文源码路径均相对下表仓库根，关键论断挂源码永久链接（permalink）：

项目	锚定 commit	永久链接	访问日期
LangGraph	main 分支 commit `3c916d7`（v1.2.19 后数个提交）	github.com/langchain-ai/langgraph/tree/3c916d7	2026-11-03
Codex CLI	main 分支 commit `bfdbf01`	github.com/openai/codex/tree/bfdbf01	2026-11-03
OpenClaw	main 分支 commit `4285c6ce`（CalVer 2026.10.10 线）	github.com/openclaw/openclaw/tree/4285c6ce	2026-11-03

文档调研工具（固定 release + 核心官方文档页）——论断出处，访问日期同上：

项目	锚定版本	核心官方文档页
LangChain	langchain 1.4.8 · langchain-core 1.7.4	`langchain/overview`（官方分层表述）| `structured-output`（结构化输出双策略）| `multi-agent`（五模式）
Pydantic-AI	v2.65.0	`api/output`（结构化输出）| `deferred-tools`（审批）| `mcp/client`（MCP）
dsh	dsh-v0.2.2-alpha.3	`docs/subsystems`（session / sandbox / approval / tools）
Pi	v1.5.0	`extensions`（扩展体系）| `sessions`（会话树）
Hermes-Agent	v0.22.6	`user-guide/security`（审批与黑名单）| `integrations/providers`（模型接入）

注册表与第三方佐证：npm / PyPI registry API（2026-11-03）；安全事件佐证——CCB Belgium 公告（ccb.belgium.be，1-click RCE advisory）、SentinelOne / Cisco / Immersive Labs 分析、arXiv:2503.03704（记忆投毒 MINJA）等。完整 URL 清单见随附调研卡 `research/*.md`。

报告生成：2026-11-03